

网络编程实验指导手册

本指导手册内容来自：《计算机网络实验教程》（第2版），王盛邦编著，清华大学出版社，ISBN：9787302461234，2017年5月

实验1 UDP 通信程序设计

【实验目的】

选择一个操作系统环境（Linux 或者 Windows），编写 UDP 网络通信程序，实现 UDP 网络通信。可以使用自己熟悉的编程语言，如 C++或者 Python。

【实验内容】

在发送 UDP 数据包时做一个循环，连续发送 100 个数据包；在接收端统计丢失的数据包。

【实验要求】

实验时，请运行 Wireshark 软件，对通信时的数据包进行跟踪分析。

【实验思考】

- （1）说明在实验过程中遇到的问题 and 解决方法。
- （2）给出程序详细的流程图和对程序关键函数的详细说明。
- （3）使用 Socket API 开发该 UDP 通信程序中的客户端程序和服务器程序时，使用了哪些不同的函数？解释 `connect()`、`bind()` 等函数中 `struct sockaddr *addr` 参数各个部分的含义，并用具体的数据举例说明。
- （4）说明面向连接的客户端和面向非连接的客户端在建立 Socket 时有什么区别。
- （5）说明面向连接的客户端和面向非连接的客户端在收发数据时有什么区别。面向非连接的客户端又是如何判断数据发送结束的？
- （6）比较面向连接的通信和无连接通信，它们各有什么优点和缺点？适合在何种场合下使用？
- （7）尝试在实验中发送更多数量的包，若有丢包，请分析引起 UDP 丢包的可能原因是什么？

实验 2 TCP 通信程序设计

【实验目的】

选择一个操作系统环境（Linux 或者 Windows），编写 TCP 通信程序，实现 TCP 网络通信。可以使用自己熟悉的编程语言，如 C++或者 Python。

【实验内容】

分别编写用于网络聊天的 TCP 服务器端和客户端两个程序。聊天使用者通过客户端程序登录到服务器，实现与登录在同一服务器上的另一个用户的网络交谈。

TCP 服务器端程序通过 `socket ()` 系统调用创建一个 Socket（设定其接受连接客户的最大数目），与指定的本地端口绑定 `bind ()`，在该端口上进行侦听 `listen ()`。如果有 TCP 客户端连接请求，则在数组中选择一个空 Socket，将客户端地址赋给该 Socket，然后该客户就可以在服务器上聊天了。

TCP 客户端程序建立一个 Socket 与服务器端连接，成功后即可通过该 Socket 发送和接收数据。

【实验要求】

- （1）客户端登录到聊天服务器，服务器管理所有登录的客户，并将客户列表发送给各个客户显示。
- （2）客户端通过服务器转发，实现一对一聊天。当客户端程序连接服务器时，通过服务器搜索其想要聊天的客户，如果检测到此用户且该用户正处于联网状态，则服务器通知此用户的客户端程序响应主叫方客户端程序，然后在主叫方和被叫方建立连接后，双方实现聊天。
- （3）客户端程序实时显示目前其他用户是否在线或者下线的状态。
- （4）实验时，请运行 Wireshark 软件，对通信时的数据包进行跟踪分析。

【实验思考】

- （1）说明在实验过程中遇到的问题和解决方法。
- （2）请用程序语句说明，服务器端如何获取连接到其上的所有客户的列表、并发送给各个用户的？服务器端程序如何获取客户端是否在线的状态？
- （3）请用程序语句说明，客户如何通过服务器转发实现一对一聊天？

附录：用 Visual C++ 编程实现 TCP 通信示例

在 Visual C++ 中进行 Winsock 的 API 编程开发时，需要使用下面 3 个文件，否则会出现编译错误。

(1) winsock.h: Winsock API 的头文件。

(2) wsock32.lib: Winsock API 连接库文件。

(3) winsock.dll: Winsock 的动态连接库，位于 Windows 的安装目录下。

程序分两部分：服务器端程序和客户端程序。

(1) 服务器端程序

```
//TCPdtd_server.cpp - main, TCPdaytimed
```

```
#include<stdlib.h>
```

```
#include<stdio.h>
```

```
#include<winsock2.h>
```

```
#include<time.h>
```

```
void errexit(const char *,...);
```

```
void TCPdaytimed(SOCKET);
```

```
SOCKET passiveTCP(const char *, int);
```

```
#define QLEN 5
```

```
#define WSVERS MAKEWORD(2, 0)
```

```
/*
```

```
 * main - Iterative TCP server for DAYTIME service
```

```
*/
```

```
void
```

```
main(int argc, char *argv[])
```

```
/* argc: 命令行参数个数，例如： C:\>TCPdaytimed 8080, argc=2 argv[0]="TCPdaytimed",  
argv[1]="8080" */
```

```
{
```

```
    struct sockaddr_in fsin;    /* the from address of a client */
```

```
    char *service = "daytime"; /* service name or port number */
```

```

SOCKET msock, ssock;      /* master & slave sockets */
int alen;                  /* from-address length */
WSADATA wsadata;          //Socket 的版本信息

switch (argc) {
case 1:
    break;                //命令行仅文件名(即命令动词)
case 2:
    service = argv[1];     //命令行仅一个参数
    break;
default:
    errexit("usage: TCPdaytimed [port]\n");//其他情况
}

if (WSAStartup(WSVERS, &wsadata) != 0)
//Winsock 服务初始化。首参数指明程序请求使用的 Socket 版本 (高位字节指明副版本,
//低位字节指明主版本); 次参数返回请求的 Socket 的版本信息
errexit("WSAStartup failed\n");

msock = passiveTCP (service, QLEN); // 创建被动的套接字, 调用 passiveTCP 实现。第
//一个是字符串: 服务的名字或者端口号; 第二个是传入连接的请求队列所需的长度
while (1) {
    alen = sizeof (struct sockaddr);
    ssock = accept (msock, (struct sockaddr *)&fsin, &alen);
    if (ssock == INVALID_SOCKET)
        errexit ("accept failed: error number % d\n", GetLastError ());
    TCPdaytimed (ssock);
    (void) closesocket (ssock); // 调用 close 是从容关闭: TCP 保证所有的数据可靠交
    // 付给客户 (连接终止前收到确认)
}
// 在循环中, 使用 accept 从主套接字得到一个连接 (accept 完成三次握手过程)

```

```
// 对于新的连接服务器调用过程 TCPdaytimed 进行处理
// 处理完毕继续循环，再次调用 accept 阻塞
// 循环实现就足够了，服务器忙时，其他的请求可以排队
} // main ()
```

```
/* -----
```

```
TCPdaytimed - do TCP DAYTIME protocol
```

```
*/
// 从其他机器上获得另一个系统当前的日期和时间
Void TCPdaytimed (SOCKET fd)
{
    char pts; / pointer to time string /
    time_t now; / current time */

    (void)time(&now);
    pts = ctime(&now);
    (void)send(fd, pts, strlen(pts), 0);
    printf("%s", pts);
}
```

(2) 客户端程序

```
/* TCPdtc_client.cpp - main, TCPdaytime */
```

```
#include<stdlib.h>
#include<stdio.h>
#include<winsock2.h>
```

```
void TCPdaytime(const char *, const char *);
void errexit(const char *, ...);
```

```
SOCKET connectTCP(const char *, const char *);
```

```
#define LINELEN 128
```

```
#define WSVERS MAKEWORD(2, 0)
```

```
/*
```

```
 * main -TCP client for DAYTIME service
```

```
 */
```

```
int
```

```
main(int argc, char *argv[]) /* usage: TCPdaytime [host [port]]\n */
```

```
{
    /* host 可以为服务器的 IP 地址或域名, port 可
       以为服务器监听的端口号或服务名 */
```

```
    char *host = "localhost"; /* host to use if none supplied */
```

```
    char *service = "daytime"; /* default service port */
```

```
    WSADATA wsadata;
```

```
    switch (argc) {
```

```
    case 1:
```

```
        host = "localhost";
```

```
        break;
```

```
    case 3:
```

```
        service = argv[2]; /* FALL THROUGH */
```

```
    case 2:
```

```
        host = argv[1];
```

```
        break;
```

```
    default:
```

```
        fprintf(stderr, "usage: TCPdaytime [host [port]]\n");
```

```
        exit(1);
```

```
    }
```

```

if (WSAStartup(WSVERS, &wsadata) != 0) /* 启动某版本的 DLL */
    errexit("WSAStartup failed\n");
TCPdaytime(host, service); /* 建立连接并传输数据 */
WSACleanup(); /* 卸载某版本的 DLL */
printf("按任意键继续...");
getchar();
return 0; /* exit */
}

/*
 * TCPdaytime - invoke Daytime on specified host and print results
 */
void TCPdaytime(const char *host, const char *service)
{
    char buf[LINELEN + 1]; /* buffer for one line of text */
    SOCKET s; /* socket descriptor */
    int cc; /* recv character count */

    s = connectTCP(host, service); /* 建立连接 */

    cc = recv(s, buf, LINELEN, 0);
    while (cc != SOCKET_ERROR && cc > 0) {
        buf[cc] = '\0'; /* ensure null-termination */
        (void)fputs(buf, stdout);
        cc = recv(s, buf, LINELEN, 0);
    }
    closesocket(s);
}

```

(3) 程序运行

为了运行程序，须先编译程序：在 Visual C++ 下建立 dsw 文件，然后生成并

运行（按 Ctrl+F5 键）。使用套接字函数的程序链接时需要加载 ws2_2.lib，即在工程设置 /Link/General/ 对象 / 库模块中加入 ws2_2.lib。

运行程序时要先运行服务器程序：在命令窗口中输入 TCPdtc_server server_ip [server_port]\n；在另一台计算机上运行 TCPdtc_client server_ip [server_port]\n。如果程序没有回应，可能是防火墙引起服务器的监听端口关闭。在程序运行时可以启动 Wireshark 捕获数据包，通过分析可以展示 C/S 的连接过程。